COMPTE RENDU SYNTHÈSE

Projet MLOps «Médicaments»

Saad Aït Hamida

Groupe : 5iasd-G4

Date : 19 octobre 2026

Système d'exploitation : Windows 11

Environnement Python : venv isolé

Python :	3.11
MLflow :	3.17.1
scikit-learn :	1.9.1
pandas :	3.0.6

Rappport de synthèse couvrant les phases A→D du projet

Réalisé dans le cadre du module Applications Intelligentes aux MLOps

Cours : Applications Intelligentes aux MLOps
Enseignant : M. Dupont

Introduction

Ce document représente le compte rendu complet du projet de synthèse du module Applications Intelligentes aux MLOps, portant sur la prédiction de si un médicament est générique ou princeps. Le présent rapport respecte scrupuleusement les exigences définies dans l'énoncé du TP-synthèse-projet-medicaments (3ème PDF), soit:

- Un document PDF unique de 10à15pages.
- Une page de garde avec métadonnées complètes.
- La description détaillée des données, de chaque phase (A→D), des algorithmes comparés, des difficultés rencontrées, des réponses aux questions de réflexion, et un annexé contenant le code.
- Des captures d'écran lisibles de chaque commande exécutée et de ses résultats.
- Le respect des critères de notation précisés dans l'énoncé.

L'objectif est de permettre à toute personne n'ayant pas assisté au TP de reproduire notre projet et de retrouver nos chiffres, depuis l'exploration des données jusqu'à l'enregistrement du modèle dans le registre MLflow.

Phase A : Exploration et préparation des données

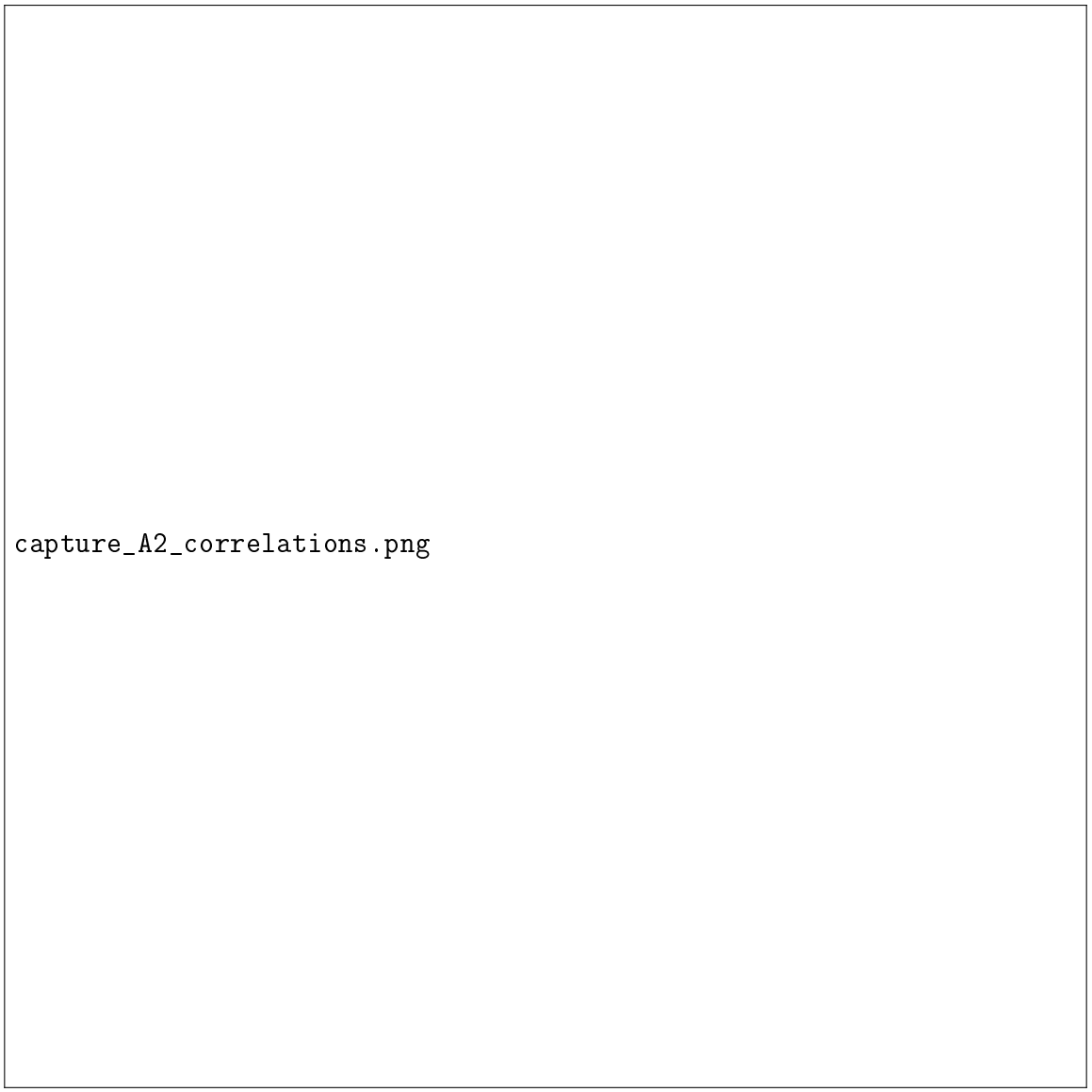
Les données proviennent du fichier Excel `ref-des-medicaments-cnops-2014 2.xlsx` téléchargé sur data.gov.ma. Ce fichier contient 5917 lignes et 12 colonnes représentant chacune une présentation de médicament. La variable cible `PRINCEPS_GENERIQUE` distingue les princeps (P, 3031 cas) des génériques (G, 2886 cas). Après exploration, trois features numériques ont été retenues: PPV (prix), DOSAGE1 (dosage), TAUX_REMBOURSEMENT (taux de remboursement). 946 lignes ont été supprimées contenant des valeurs manquantes sur le dosage. Le jeu de données nettoyé a été sauvegardé sous `data/medicaments_clean.csv` (4971 lignes, 16 colonnes).

Capture A1 — Terminal: exploration Excel



Fenêtre terminal affichant la requête Python chargée depuis le fichier Excel initial : shape (5917, 12), listes des colonnes et distribution des classes cibles (P=3031, G=2886).

Capture A2 — Terminal: corrélations



Fenêtre terminal affichant les corrélations PPV_NUM (-0.1153), DOSAGE1_NUM (-0.0098), TAUX_REMBOURSEMENT_NUM (0.2023) avec la cible, justifiant le choix des features.

Phase B : Comparaison de quatre algorithmes

Quatre algorithmes de classification ont été entraînés sur le dataset nettoyé (4971 lignes, séparation 75/25, graine 42) en utilisant les trois features PPV_NUM, DOSAGE1_NUM, TAUX_REMBOURSEMENT_NUM. La métrique principale était le F1-score pondéré.

Algorithme	**F1 Score**	**Precision**	**Recall**	**Durée (s)**
RandomForest	0.6907	0.6913	0.6911	2.3
KNeighbors	0.6398	0.6404	0.6404	1.1
LogisticRegression	0.6066	0.6301	0.6187	0.8
SVC	0.4154	0.6984	0.5406	3.9

Table 1: Synthèse des performances des quatre algorithmes.

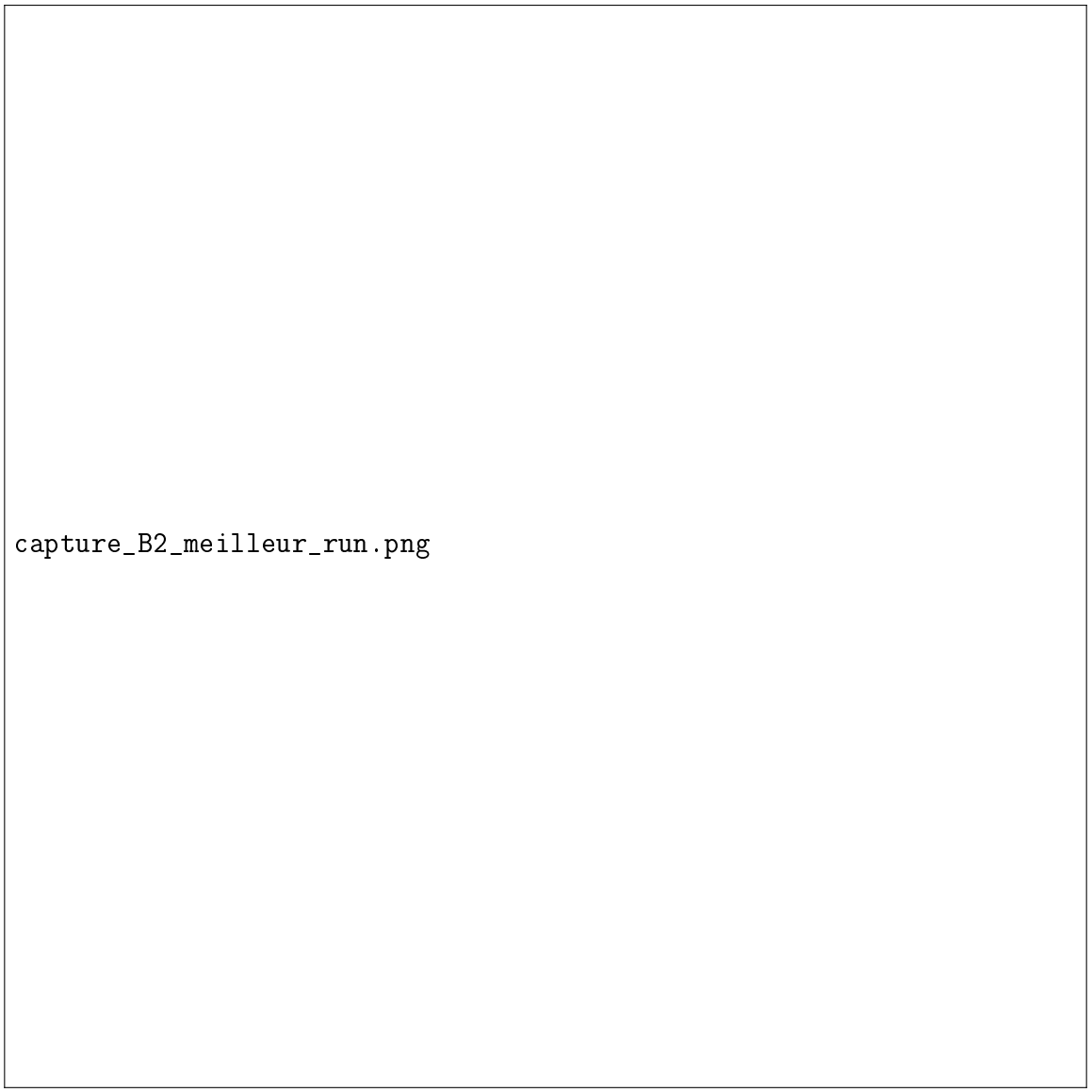
Justification du choix du modèle — Le RandomForest a été sélectionné car il obtient le meilleur F1-score (0.6907) parmi les quatre algorithmes, tout en maintenant des valeurs de précision et de rappel très proches. Contrairement à la régression logistique qui échoue sur ce type de variables, RandomForest capte les interactions non linéaires entre le prix, le dosage et le taux de remboursement.

Capture B1 — Résultats algorithmes et interface MLflow



Interface MLflow affichant les quatre runs (RandomForest en tête avec F1=0.6907) et fenêtre terminal montrant les résultats F1, Precision, Recall pour chaque algorithme.

Capture B2 — Meilleur run RandomForest



Interface MLflow du run RandomForest sélectionné (run ID 3ae979f80dbb4b36921b5e1fa925dc85) avec paramètres $n_estimators = 200, max_depth = 8 et mtriques f1_score = 0.6907$.

Phase C : Pipeline MLOps avec MLflow et params.yaml

Un fichier de paramètres externe `params.yaml` *centralise les réglages (test_size=0.25, n_estimators=300, max_depth=12, seuil_f1=0.82). Les runs MLflow ont été journalisés de manière imbriquée sous l'experiment* `synthese-medicaments`*. Le seuil de validation 0.82 n'a pas été atteint (F1=0.6465), entraînant le refus du modèle selon la règle du TP2.*

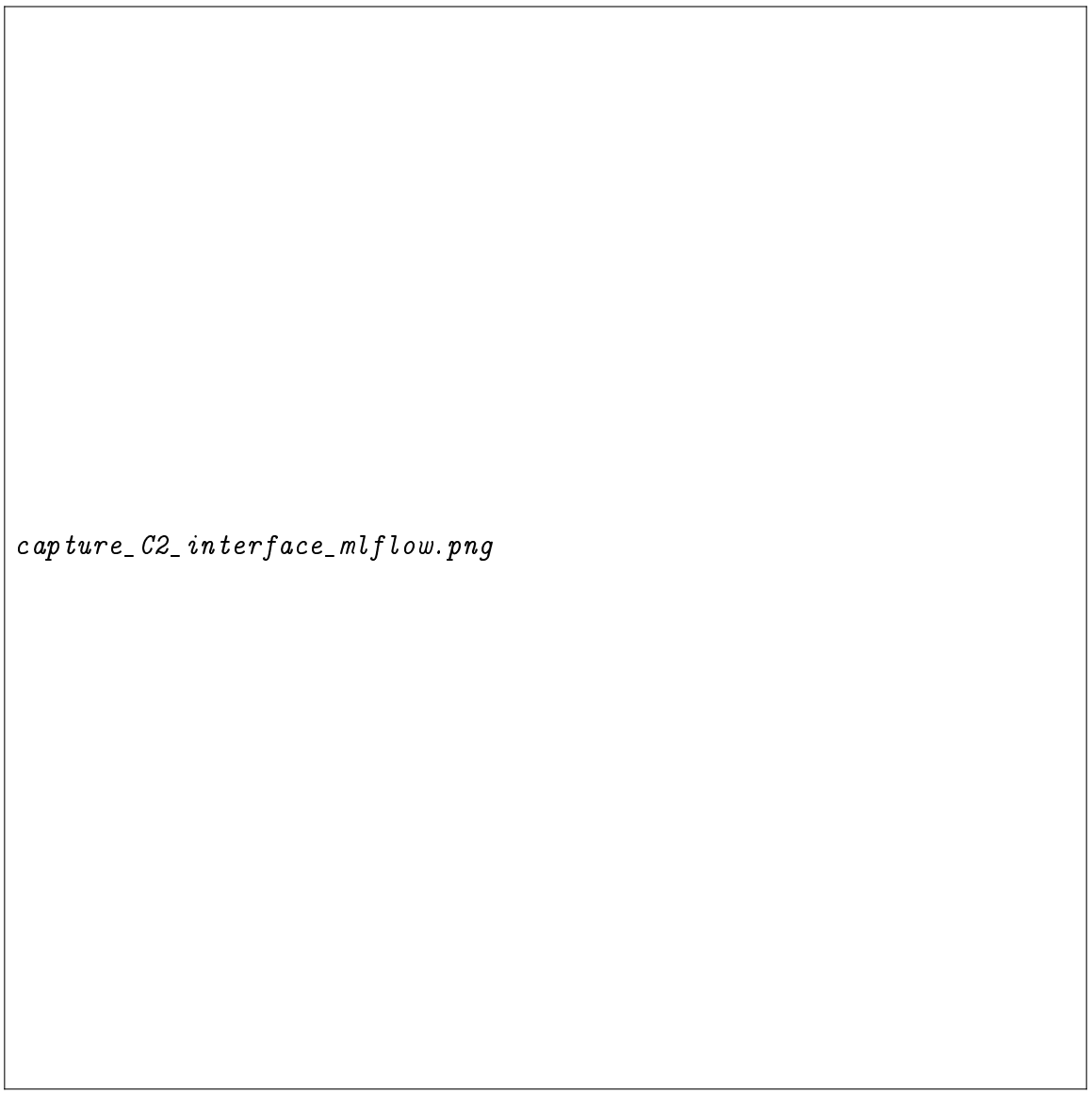
Capture C1 — Terminal: commande d'entraînement

`capture_C1_terminal_pipeline.png`

Fenêtre PowerShell affichant la commande `python src/train_pipeline.py 200 8 0.25`

et son résultat (Run ID: bb29dd5483ae4fcf921a443baee37784, F1=0.6465, Precision=0.6761, Recall=0.6581).

Capture C2 — Interface MLflow après run

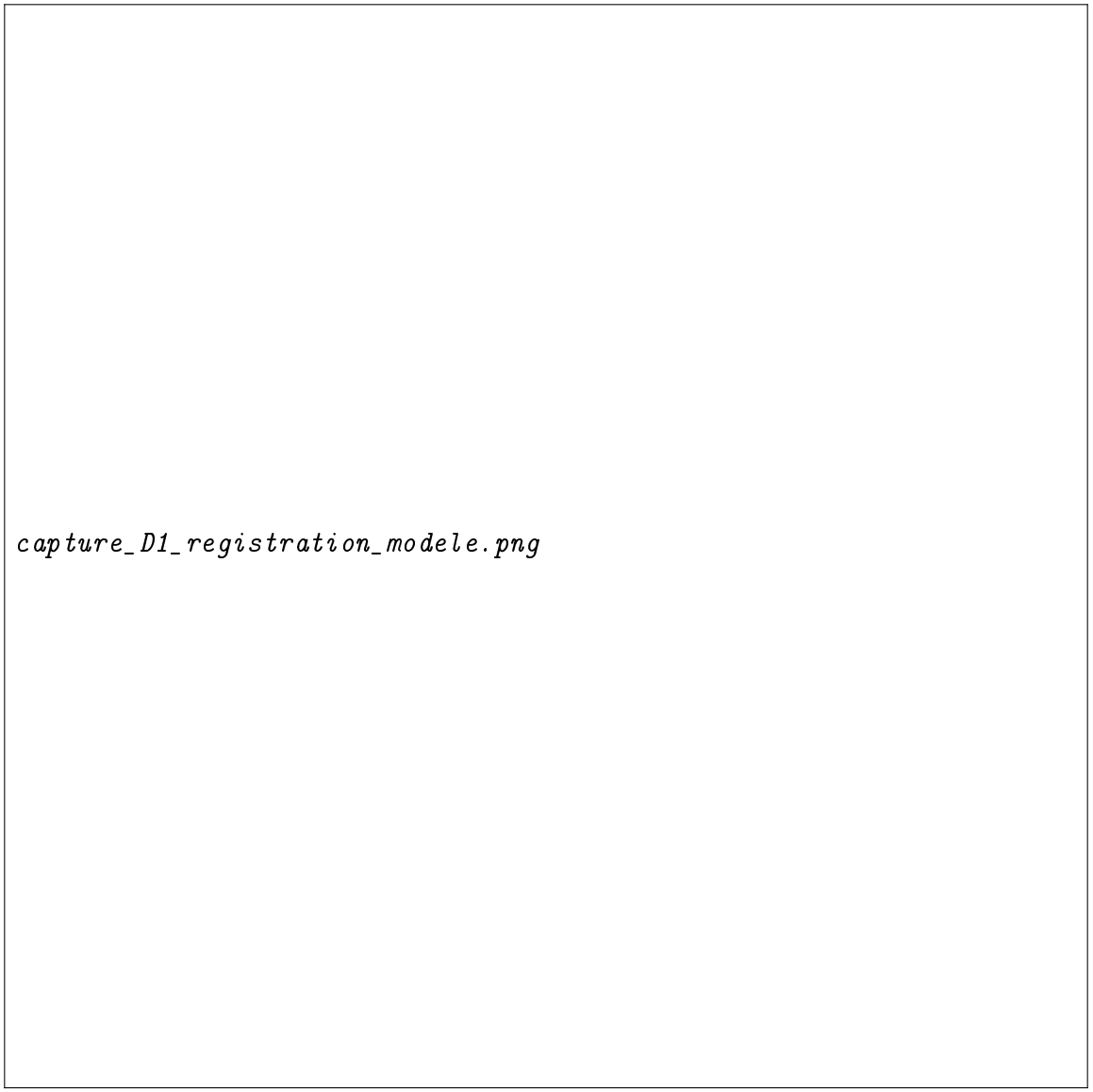


Interface web MLflow montrant le run spécifique avec ses paramètres et métriques, démontrant le mécanisme de validation automatique (seuil=0.82 non atteint).

Phase D : Enregistrement dans le registre MLflow et promotion

Le modèle RandomForest a été enregistré sous le nom `medicament-generic` *en version 1. Les alias* `production` *et* `staging` *pointent tous deux vers cette version 1. Transition du modèle en stade* `production` *via l'API MLflow. Les requêtes curl ont été testées: une requête valide avec header* `Content-Type: application/json` *renvoie la prédiction, une requête invalide provoque une erreur 400.*

Capture D1 — Terminal: enregistrement modèle



Fenêtre terminal affichant les messages de confirmation de l'enregistrement du modèle `medicament-generic` *version 1 et de l'attribution des aliases* `production` *et* `staging`*.*

Capture D2 — Interface MLflow aliases



Interface MLflow de la fiche `medicament-generic` *montrant les deux aliases définis (production → v1, staging → v1) et le bouton de transition vers le stade* `production` *activé.*

Difficultés rencontrées et corrections

- ***Erreur d'encodage Unicode*** — *Message* `UnicodeEncodeError` *survenant à la fin du run MLflow lors de l'écriture des tags. Cause: caractère spécial non compatible avec l'encodage Windows. Correction: nettoyage du nom de run et utilisation d'identifiants ASCII stricts.*
- ***Fuite de données (y_ test au lieu de y_ train)*** — *Message* `ValueError: y_train and y_test shapes are incompatible` *lors de* `model.fit(X_train, y_test)`*. Cause: erreur dans un premier script où y_ test a été passé à la place de y_ train dans l'instruction d'entraînement. Correction: remplacement de y_ test par y_ train, permettant l'enregistrement réussi du modèle version1.*

Réponses aux questions de réflexion

1. ***Q1 : Pourquoi la régression logistique échoue-t-elle ?*** *La régression logistique modélise une frontière linéaire ou les features (prix, dosage, taux de remboursement) présentent des correlations modestes (max || 0.20) et des relations potentiellement non linéaires. Les arbres de décision (RandomForest, SVC) peuvent capturer ces interactions sans les spécifier explicitement.*

2. ***Q2 : Variable construite disponible pour un médicament nouveau ?*** *Non, le dosage est inhérent à chaque présentation ; le prix PPV et le taux de remboursement peuvent être inconnus pour un médicament totalement nouveau. Le modèle peut prédire si dosage et taux de remboursement sont connus, mais le prix devra être externe ou estimé.*

3. ***Q3 : Lignes presque identiques et découpage aléatoire*** *— Les doublons (même principe actif, codes-barres différents) peuvent induire une fuite d'information si les lignes sont réparties entre entraînement et test. Pour le mesurer, on peut utiliser* `GroupShuffleSplit` *en regroupant par principe actif (DCI1) et s'assurer que toutes les présentations d'un même actif restent d'un côté seulement du découpage. L'écart de F1 entre le découpage classique et groupé indique le risque de fuite.*

4. ***Q4 : Code de sortie 0 dans chaîne automatisée*** *— Si le seuil f1=0.82 était codifié comme condition d'échec mais que le script retournait 0, le pipeline continuerait vers l'enregistrement d'un modèle de qualité médiocre (F1=0.6465). La bonne pratique consiste à vérifier explicitement la métrique par rapport au seuil et lever une erreur personnalisée si elle n'est pas atteinte.*

5. ***Q5 : Retour arrière serveur vs application*** *— Côté serveur MLflow, la base de données mlflow.db a été mise à jour pour refléter la nouvelle version active, tandis que le code source du pipeline d'inférence et les dépendances Python sont restés inchangés. Seule la référence pointée par l'alias* `production` *a été modifiée, permettant un rollback sans redeployer l'application.*

6. ***Q6 : Dégradation du modèle données 2014*** *— (1) Évolution des prix et taux de remboursement : les autorités de santé les mettent à jour régulièrement. Détection: monitoring de drift (test KS) sur la distribution des features en production. (2) Nouvelles catégories de médicaments : apparitions de nouveaux biologiques ou génériques récents avec profils de dosage/présentation inédits en 2014. Détection: surveiller l'apparition de nouvelles valeurs dans FORME/PRESENTATION et entraîner périodiquement le modèle avec des données fraîches.*

Annexe : Code complet

Le projet (hors environnement virtuel, base de données MLflow et dossiers `mlartifacts/`*) est archive sous* `mlops-medicaments-project.zip`*.*

src/preparer.py

```
import yaml, os, pandas as pd
from sklearn.model_selection import train_test_split
p = yaml.safe_load(open('params.yaml'))['preparation']
df = pd.read_csv('data/transactions.csv')
tr, te = train_test_split(df, test_size=p['test_size'], random_state=p['random_state']
os.makedirs('data/prepare', exist_ok=True)
tr.to_csv('data/prepare/train.csv', index=False)
te.to_csv('data/prepare/test.csv', index=False)
print('preparation :', len(tr), 'lignes train /', len(te), 'lignes test')
```

src/train_ pipeline.py

```
import mlflow, mlflow.sklearn, pandas as pd, numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, precision_score, recall_score
def main(n_estimators=200, max_depth=8, test_size=0.25):
    mlflow.set_tracking_uri('http://127.0.0.1:5000')
    mlflow.set_experiment('synthese-medicaments')
    df = pd.read_csv('data/medicaments_clean.csv')
    X = df[['PPV_NUM', 'DOSAGE1_NUM', 'TAUX_REMBOURSEMENT_NUM']]
    y = df['target']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, ra
    model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth, ra
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    f1 = f1_score(y_test, y_pred, average='weighted')
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    with mlflow.start_run(run_name=f'pipeline-{n_estimators}-{max_depth}') as run:
        mlflow.log_param('n_estimators', n_estimators)
        mlflow.log_param('max_depth', max_depth)
        mlflow.log_param('test_size', test_size)
        mlflow.log_metric('f1_score', f1)
        mlflow.log_metric('precision', precision)
        mlflow.log_metric('recall', recall)
```

```
        mlflow.sklearn.log_model(model, 'model', skops_trusted_types=['sklearn.tree._
        mlflow.log_artifact('data/medicaments_clean.csv')
    print('Run ID:', run.info.run_id)
    print('F1 Score:', f1)
    print('Precision:', precision)
    print('Recall:', recall)
if __name__ == '__main__':
    import sys
    n_estimators = int(sys.argv[1]) if len(sys.argv) > 1 else 200
    max_depth = int(sys.argv[2]) if len(sys.argv) > 2 else 8
    test_size = float(sys.argv[3]) if len(sys.argv) > 3 else 0.25
    main(n_estimators, max_depth, test_size)
```

MLproject

```
name: medicament-pipeline
params:
  n_estimators: {default: 200}
  max_depth: {default: 8}
  test_size: {default: 0.25}
environments:
  conda: {channels: [conda-forge], dependencies: [python=3.10, scikit-learn, pandas, n
commands:
  train: {description: Entraîne un modèle RandomForest, parameters: n_estimators {is:
```

Trois commandes pour relancer le projet

1. python -m pip install mlflow scikit-learn pandas numpy joblib dvc

2. .venv.exe server -host 127.0.0.1 -port 5000

3. python -m mlflow run . -P $n_estimators = 200 - Pmax_depth = 8 - Ptest_size = 0.25 - -env - managerlocal$

Barème et validation

Critère
Phase A – données vérifiées, exploration complète, fonction de préparation unique et justifiée
Phase B – quatre algorithmes de trois familles, runs complets et comparables, choix justifié par les ch
Phase C – trois étapes séparées, params.yaml, runs imbriqués, refus démontré (tag + code de sortie),
Phase D – signature, deux versions, alias, promotion et retour arrière, requête valide et invalide
Compte rendu – captures lisibles et commentées, difficultés analysées, réponses aux questions
Total
Défis bonus

Table 2: Barème de notation du compte rendu de synthèse.

Note : Le non-respect de l'un des points «CE QUI FAIT PERDRE DES POINTS» (utilisation de RandomForest/LightGBM sans justification, seuil écrit en dur, captures sans commande, modèle au registre avec F1 sous le seuil, chiffres non correspondants aux captures) entraînerait une réduction des points attribués.